PROYECTO DE AULA · DESARROLLO INFORMÁTICO · UNIVERSIDAD ECCI

Water Controller Smart

Gestión inteligente del consumo de agua en los hogares de la localidad de Chapinero

INTEGRANTES DEL GRUPO
Alan Sneider Solorzano
Juan Sebastian Roa Benavides
Jhon Edinson Lasso Gomez

CURSO
Electiva (Python) - 5AML

DOCENTE
Alied Victoria Garzon Gordo

FECHA
13/11/2026

02 Introducción

Problema que resuelve

En los hogares de Chapinero no hay herramientas simples para medir el tiempo y el volumen de agua que se gasta en tareas diarias, lo que genera un desperdicio constante.

Motivo del tema

Falta concientización sobre los hábitos de consumo. Medirlos permite adquirir hábitos sostenibles y reducir el valor de la factura de servicios públicos.

Objetivo general

Desarrollar una aplicación web que estime el agua gastada según el tiempo de cada actividad y genere alertas preventivas ante excesos.

Alcance del sistema

Plataforma web para registrar actividades (ducha, platos, riego, aseo), calcular litros, seguir un límite diario y recibir alertas en la web y por correo.

03 Requerimientos

Funcionales

RF-01	**Registrar / cronometrar actividad**	Cronómetro en vivo o minutos manuales
RF-02	**Cálculo de agua gastada**	Litros = minutos × caudal
RF-03	**Evaluación de excesos**	Tiempo por actividad y meta diaria
RF-04	**Visualización por categorías**	Litros y % por tipo de actividad
RF-05	**Configuración de actividades**	Caudal (L/min) personalizable
RF-06	**Configuración de metas diarias**	Límite diario en litros con alertas

No funcionales

RNF-01	**Usabilidad**	Interfaz sencilla e intuitiva
RNF-02	**Rendimiento**	Procesa y muestra datos rápido
RNF-03	**Seguridad**	Evita accesos no autorizados
RNF-04	**Disponibilidad**	Con energía y conectividad
RNF-05	**Escalabilidad**	Nuevas categorías de actividad
RNF-06	**Confiabilidad**	Caudales validados o personalizados

04 Historias de Usuario

Como [usuario], quiero [acción], para [beneficio]

US-001 **Alta**

Quiero registrar mis actividades con cronómetro o minutos, **para** controlar el tiempo que uso el agua.

US-002 **Alta**

Quiero que el sistema calcule los litros gastados por actividad, **para** saber cuánta agua consumo.

US-003 **Alta**

Quiero que detecte duraciones largas o metas superadas, **para** identificar excesos y corregir hábitos.

US-004 **Alta**

Quiero recibir alertas en la web y por correo, **para** corregir mi consumo a tiempo.

US-005 **Alta**

Quiero ver mi consumo por tipo de actividad, **para** saber en qué tareas gasto más agua.

US-006 **Media**

Quiero consultar mi historial y litros estimados, **para** verificar si reduzco el desperdicio.

US-007 **Media**

Quiero personalizar caudal y tiempo recomendado, **para** ajustar las estimaciones a mis grifos.

US-008 **Media**

Quiero fijar un límite diario en litros, **para** recibir alertas cuando me acerque al tope.

05 Metodología Ágil

Scrum / Kanban

Elegimos Scrum frente a Kanban: seis sprints con entregas incrementales, revisión y retrospectiva al cierre de cada uno.

Roles del equipo

Jhon Edinson Lasso Gomez
Desarrollador · arquitectura

Alan Sneider Solorzano
QA · calidad y criterios

Juan Sebastian Roa Benavides
Tester · pruebas e integración

Backlog

Product Backlog con 8 historias de usuario (US-001 a US-008), priorizadas y estimadas con Planning Poker, más tres ítems técnicos: UNI-001 (integración), MIG-001 (migración) y QA-001 (pruebas).

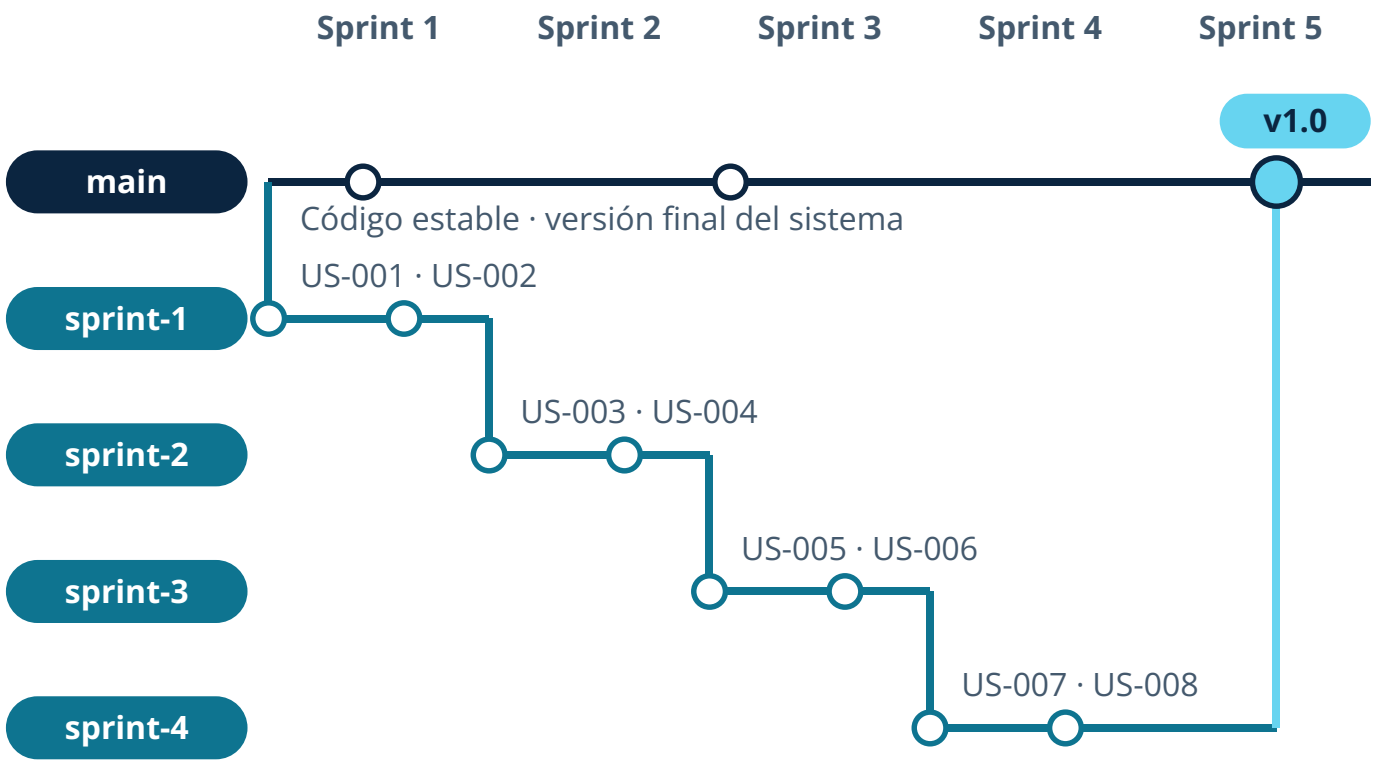
SPRINTS · 23 DE SEPTIEMBRE AL 13 DE NOVIEMBRE DE 2026

Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5	Sprint 6
23 sep – 2 oct	3 – 10 oct	10 – 17 oct	17 – 24 oct	25 – 31 oct	1 – 13 nov
Registro y cálculo del consumo	**Excesos y alertas de consumo**	**Historial, desglose y reportes**	**Caudales y límites a medida**	**Integración final · Release v1.0**	**Flask + SQLite y pruebas**
US-001 · US-002	US-003 · US-004	US-005 · US-006	US-007 · US-008	UNI-001	MIG-001 · QA-001

06 Manejo de versiones

METODOLOGÍA ÁGIL · CONTROL DE VERSIONES CON GIT

RAMAS Y FLUJO DE VERSIONES · GIT / GITHUB



Sprint 5 · UNI-001: merge de sprint-4 a main · **Sprint 6:** Flask + SQLite

Repositorio en GitHub

Repositorio público del equipo en GitHub. La rama main guarda el código estable de la aplicación.

github.com/jelg17/water-controller-smart

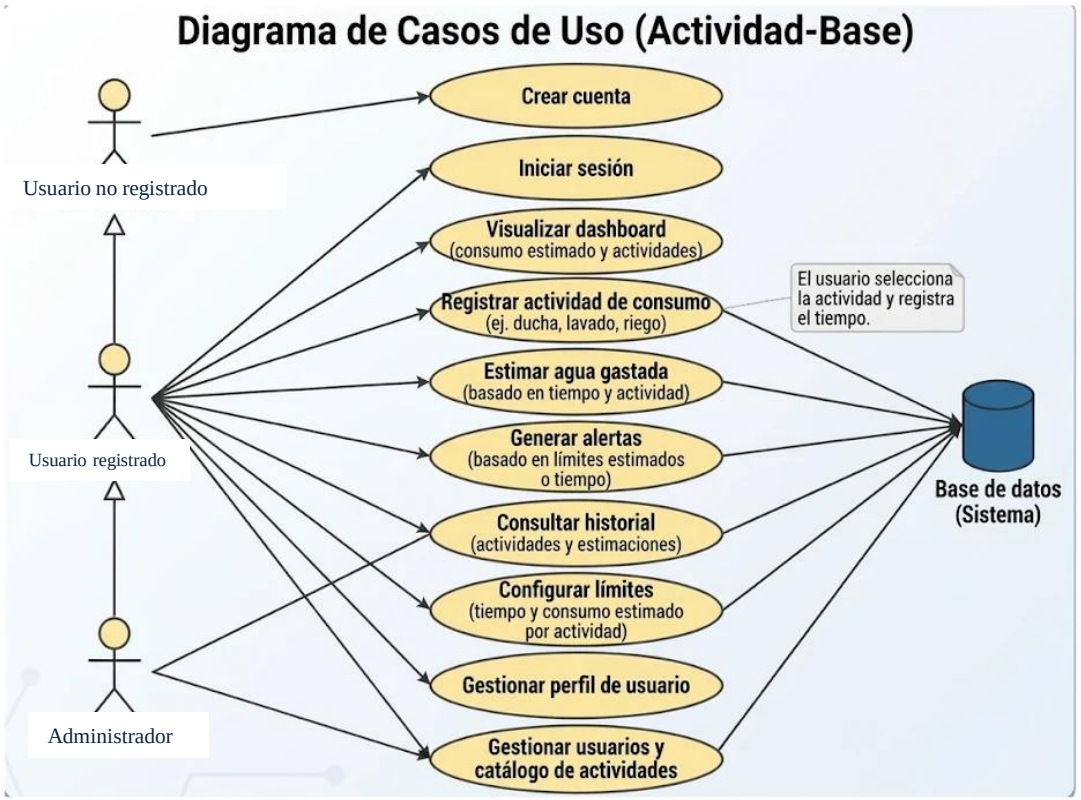
Una rama por sprint

Cada sprint avanzó en su propia rama. En el Sprint 5 sprint-4 se fusionó a main, resolviendo conflictos en servidor.py y datos.json.

Releases v1.0 y v2.0

Tras las pruebas E2E se publicó la v1.0 (Sprint 5). El Sprint 6, con Flask + SQLite y las pruebas, cierra con la v2.0 final.

07 Casos de Uso



Actores: usuario no registrado, usuario registrado, administrador y base de datos.

Iniciar sesión
Acceso con correo y contraseña

Registrarse
Crear una cuenta nueva

Monitorear consumo de agua
Cronómetro o minutos manuales

Detectar desperdicios
Compara tiempo y meta diaria

Generar alertas
Aviso por web y correo

Consultar historial
Gráficos y tablas por fechas

Configurar límites
Tope diario o mensual en litros

Visualizar dashboard
Consumo, métricas y alertas activas

08 Diagramas de Clase

CLASES PRINCIPALES

Usuario, Actividad, SesiónConsumo, Configuración, Alerta y Notificación.

ATRIBUTOS Y MÉTODOS

Los datos que guarda cada clase.

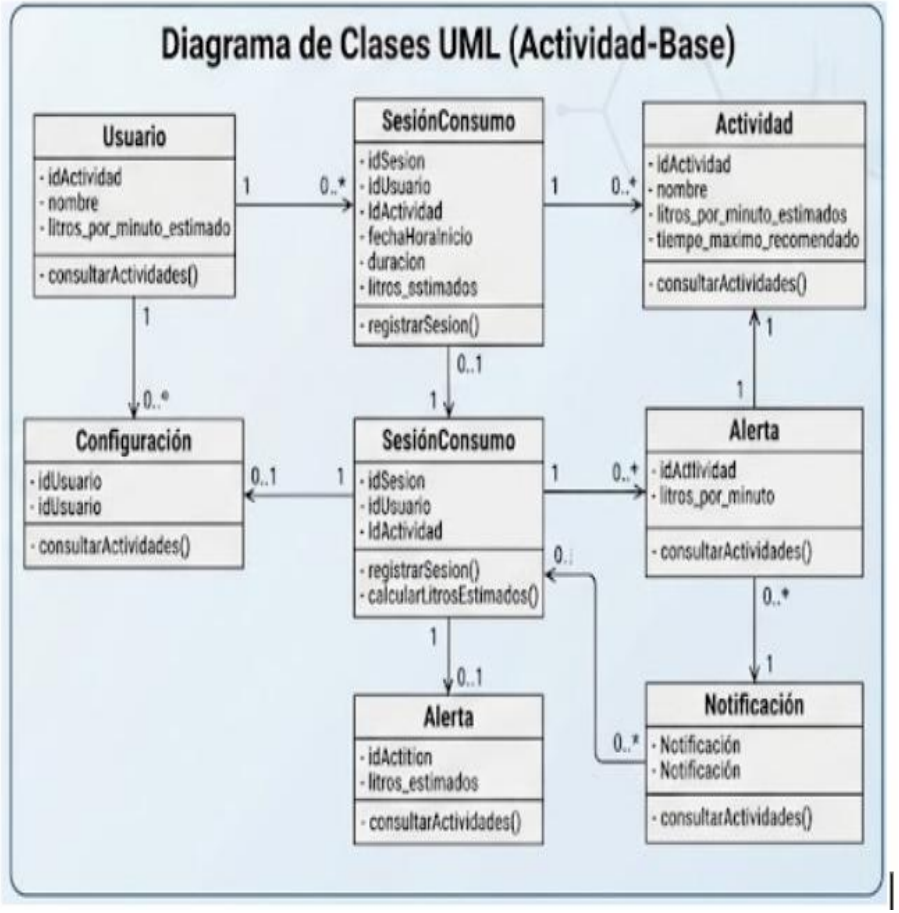
Actividad: nombre, litros_por_minuto, tiempo_recomendado.

SesiónConsumo: fechaHoraInicio, duración, litros_estimados.

Las acciones que realiza cada clase.

SesiónConsumo: registrarSesion(), calcularLitrosEstimado().

Actividad: calcularLitros(), calcularAhorro().



HERENCIA

Una clase hija toma los atributos y métodos de su clase base.

Actividad es la base: Ducha, LavadoPlatos, Riego y Aseo la heredan.

Notificación es base de NotificaciónWeb y NotificaciónCorreo.

POLIMORFISMO

Un mismo método responde distinto según la clase.

calcularLitros(): cada actividad aplica su caudal.

enviar(): muestra el banner web o envía el correo.

09 Demostración Python

FLASK · SQLITE · APLICACIÓN WEB EN LOCALHOST:7500

EJECUCIÓN LOCAL

```
git clone <repositorio>
pip install -r requirements.txt
python app.py
→ http://localhost:7500
```

TECNOLOGÍAS

Python 3 con Flask · SQLite · HTML, CSS y JavaScript sin frameworks · SMTP para el correo

Menú principal

Inicio de sesión y registro, panel del día con cronómetro y menú lateral: Panel, Historial, Configuración y Alertas.

CRUD

Registrar actividades con cronómetro o minutos, y crear, editar y eliminar actividades propias con su caudal.

Reportes

Historial por periodo con gráficas, comparación con el periodo anterior, ahorro potencial y exportación a CSV.

FLUJO DE LA DEMOSTRACIÓN

1. **Registrarse e iniciar sesión**
2. **Cronometrar una ducha**
3. **Ver litros, semáforo y alerta**
4. **Consultar historial y CSV**
5. **Ajustar caudal y meta**

10 Pruebas

MATRIZ DE 20 CASOS · INTERFAZ, VALORES LÍMITE, HTTP Y BASE DE DATOS

20	15	5	≥ 90 %
casos de prueba	prioridad Alta	prioridad Media	éxito total exigido

Casos normales

CP-01 · El cronómetro en vivo guarda la actividad y lo confirma

CP-04 · Cálculo: ducha 10 min = 90 L, platos 5 min = 30 L

CP-10 · Desglose por actividad (75 % y 25 %) y barra de progreso

CP-13 · Exportación CSV con tildes y totales exactos

Casos límite

CP-02 · Duración mínima válida de 0,1 min

CP-06 · Ducha de 5 min no es exceso; de 6 min sí

CP-07 · Semáforo con meta de 100 L: 78 L verde, 84 L amarillo, 108 L rojo

CP-12 · Filtros de fecha en los extremos del rango

Manejo de errores

CP-03 · Se rechazan valores vacíos, 0, −5 y texto

CP-09 · Si falla el correo, la app sigue y muestra el aviso

CP-17 · Ruta inexistente responde 404; base caída, 503

CP-19 · API: 401 sin sesión y 400 con duración inválida

CRITERIO DE SALIDA 100 % de los casos de prioridad Alta aprobados y al menos 90 % del total

11 Dificultades

Problemas encontrados

Avisos por correo
El envío SMTP planeado en el Sprint 2 (US-004) no se logró configurar a tiempo.

Conflictos al fusionar
Unir sprint-4 con main generó conflictos en servidor.py y datos.json.

Cambio de arquitectura
En el Sprint 6 se pasó de datos.json a Flask y SQLite.

Soluciones

Correo configurable
Las credenciales SMTP van en variables de entorno (.env), fuera del código.

Merge resuelto
Conflictos resueltos a mano y validados con pruebas E2E de los 4 módulos.

Base relacional
SQLite con tablas por usuario (db.py) en lugar del archivo plano.

Aprendizajes

Integraciones externas
Probarlas desde el inicio del proyecto.

Canal alterno
Si el correo falla, la alerta igual se guarda y se ve en el banner web.

Integrar seguido
Fusionar las ramas con más frecuencia evita conflictos grandes.

ESTADO ACTUAL

Las alertas se entregan en la web y por correo electrónico; si el correo no está configurado o falla, la alerta queda guardada y visible en la interfaz.

12 Conclusiones

Logros

Aplicación web funcional: registro con cronómetro o manual y cálculo automático de litros.

Semáforo, alertas en la web y por correo, historial con CSV y metas personalizables.

Seis sprints Scrum, código en GitHub y 20 casos de prueba diseñados.

Aprendizajes

Trabajar por sprints con roles claros: desarrollo, QA y pruebas.

Usar ramas y merge en Git sin perder el trabajo del equipo.

Probar valores límite y errores antes de entregar.

Mejoras futuras

Desplegar con un servidor WSGI y HTTPS; hoy corre en modo desarrollo local.

Cambiar SQLite por una base de datos para varios usuarios a la vez.

Informar la causa exacta cuando falla el envío del correo.

RESULTADO

Water Controller Smart ayuda a los hogares de Chapinero a medir cuánta agua gastan y a corregir sus hábitos a tiempo.